

Informe de cierre del proyecto

ModuShield

Implementación del módulo y seguridad, pipeline CI/CD, pruebas de seguridad, calidad del código y evaluación final

Proyecto	ModuShield - API Gateway de seguridad
Tecnología	Java 17, Spring Boot, Spring Cloud Gateway, Maven, Docker Compose
Módulos	gateway-service y demo-api
Repositorio	github.com/EvenWorseGroup/Modusheld
Documento	Informe de cierre para los puntos 1 a 4 de la rúbrica
Fecha	27 de septiembre de 2026

Conclusión general: el proyecto implementa un módulo funcional protegido con JWT y roles, ejecuta una canalización CI/CD completa con despliegue efímero verificable, supera el gate de cobertura por módulo y cuenta con análisis terminados de SonarQube y OWASP ZAP acompañados de correcciones concretas.

Resumen ejecutivo

ModuShield es un prototipo académico de gateway de seguridad que expone una API pública en el puerto 8080 y mantiene la API de demostración aislada dentro de Docker. El trabajo se completó de manera incremental: primero se consolidó el módulo de productos y su seguridad; después se establecieron pruebas unitarias y cobertura; posteriormente se automatizaron construcción, despliegue y pruebas E2E; finalmente se ejecutaron análisis estático y dinámico con SonarQube y OWASP ZAP, interpretando y corrigiendo hallazgos relevantes.

El resultado no depende de una demostración paralela: registro, login, autorización y CRUD se aplican directamente sobre la arquitectura existente de gateway-service y demo-api. Los secretos se suministran mediante variables de entorno; demo-api no se publica al host; y el workflow crea credenciales efímeras para cada ejecución.

Matriz de cumplimiento

Punto	Resultado	Evidencia principal	Estado
1. Módulo y seguridad	Productos CRUD, registro, login JWT, roles USER/ADMIN y rechazos 401/403.	Código, pruebas JUnit 5 y demostración E01-E06.	CUMPLIDO
2. Pipeline CI/CD	Prueba, cobertura, empaquetado, Docker, despliegue temporal, E2E, artefactos y teardown.	.github/workflows/ci.yml y runs exitosos.	CUMPLIDO
3. Seguridad y calidad	SonarQube Community Build y OWASP ZAP ejecutados, interpretados y acompañados de correcciones.	PDF SonarQube y reportes ZAP inicial/final.	CUMPLIDO
4. Cierre	Resultados, decisiones, problemas, limitaciones, aprendizajes y conclusión documentados.	Este informe y el inventario del entregable.	CUMPLIDO

Método de verificación

- Revisión del código Java, configuración Maven, Docker Compose y workflow de GitHub Actions.
- Inspección de reportes JaCoCo, resúmenes JUnit y evidencia E01-E12.
- Lectura del tablero y análisis documentado de SonarQube Community Build.
- Comparación de los reportes OWASP ZAP antes y después de la corrección.
- Verificación de ejecuciones exitosas del pipeline y de los artefactos preservados.

1. Implementación del módulo y seguridad

El módulo funcional elegido es la administración de productos. Su API REST se mantiene en demo-api, mientras gateway-service concentra autenticación, autorización y políticas. Las operaciones se consumen exclusivamente a través de ModuShield en localhost:8080; el backend no publica el puerto 8081 al host.

Operaciones y permisos

Operación	USER	ADMIN	Sin token
GET /api/products	Permitida	Permitida	401 Unauthorized
GET /api/products/{id}	Permitida	Permitida	401 Unauthorized
POST /api/products	403 Forbidden	Permitida	401 Unauthorized
PUT /api/products/{id}	403 Forbidden	Permitida	401 Unauthorized
DELETE /api/products/{id}	403 Forbidden	Permitida	401 Unauthorized

Autenticación y manejo de credenciales

- POST /auth/register crea usuarios USER y valida username y contraseña.
- POST /auth/login valida BCrypt y devuelve un JWT firmado con identidad, rol, emisión y expiración.
- JWT_SECRET, credenciales ADMIN y API key se leen desde variables de entorno; los valores reales no se almacenan en el repositorio.
- ADMIN se crea al iniciar el gateway a partir de ADMIN_USERNAME y ADMIN_PASSWORD.
- Los productos aceptan JWT Bearer; la ruta heredada de órdenes conserva X-API-Key sin sustituir el control de productos.

Controles complementarios

Control	Comportamiento implementado
Allowlist de rutas y métodos	Rechaza rutas no autorizadas con 403 y métodos no permitidos con 405.
Tamaño	Límite predeterminado de 8192 bytes; exceso devuelve 413.
Rate limit	Cinco solicitudes por diez segundos e identidad; la sexta devuelve 429.
Auditoría	Registra request ID, decisión, regla, estado y duración sin guardar JWT ni API key completa.
Errores de upstream	Convierte indisponibilidad del backend en 502 con respuesta uniforme.
Encabezado global	Establece X-Content-Type-Options: nosniff en respuestas locales y proxyficadas.

Pruebas del módulo

JUnit 5 cubre creación de usuarios, login, JWT, autorización, API key, rutas y métodos, límites, auditoría, manejo de errores y el comportamiento de demo-api. JaCoCo se aplica por separado a cada módulo, de modo que un resultado alto en uno no puede compensar baja cobertura en el otro.

Módulo	Pruebas archivadas	Líneas cubiertas	Total	Cobertura	Gate
gateway-service	80	449	492	91.26%	>=80% aprobado
demo-api	10	53	55	96.36%	>=80% aprobado

Estos porcentajes pertenecen al artefacto JaCoCo preservado del run 36291416738. La versión actual añadió dos pruebas de integración del encabezado de seguridad, por lo que la suite quedó en 82 pruebas del gateway y 10 de demo-api; el run 36311708560 volvió a aprobar el gate por módulo.

Demostración funcional

El runner client-tests/run_demo.py integra el comportamiento real en doce escenarios: salud, login ADMIN, JWT ausente o inválido, ruta bloqueada, permisos USER, rate limit, límites de 8192/8193 bytes, upstream 502, aislamiento de red y auditoría sin exposición del token. La evidencia conservada registra 12 aprobados, 0 fallidos y 0 omitidos.

Grupo	Escenarios	Resultado
Autenticación y autorización	E02-E06	PASS
Políticas de tráfico	E07-E09	PASS
Resiliencia y aislamiento	E10-E11	PASS
Auditoría segura	E12	PASS

2. Pipeline de CI/CD

El workflow CI and test deployment se ejecuta automáticamente en cada pull request, en cada push a main y de forma manual mediante workflow_dispatch. Utiliza un runner Ubuntu hospedado por GitHub y Java Temurin 17 con caché Maven.

Orden	Etapas	Criterio de aprobación
1	Pruebas JUnit 5	Todas las pruebas terminan sin fallos.
2	JaCoCo	Cada módulo alcanza al menos 80% de cobertura de líneas.
3	Empaquetado Maven	Se generan los JAR de gateway-service y demo-api.
4	Construcción Docker	Se construyen ambas imágenes con los Dockerfiles existentes.
5	Despliegue de prueba	Docker Compose inicia la topología aislada y /health queda listo.
6	E2E E01-E12	Los doce escenarios terminan en PASS.
7	Evidencia	Se cargan JaCoCo y JSON E2E como artefactos.
8	Teardown	Compose se elimina con if: always(), incluso después de fallos.

Entrega automatizada al entorno de prueba

El despliegue requerido por la rúbrica se implementa como un entorno efímero en el propio runner de GitHub Actions. Esta opción es compatible con Docker Compose, no exige un servidor permanente y mantiene la secuencia obligatoria: ninguna construcción o entrega ocurre si fallan pruebas o cobertura. El entorno existe durante la ejecución, se valida a través de localhost:8080 y se destruye al finalizar.

Run	Uso en la entrega	Resultado
36291416738	Evidencia descargada: JaCoCo, E2E, capturas y metadatos.	Success
36311708560	Verificación posterior con filtro de seguridad y pruebas nuevas.	Success

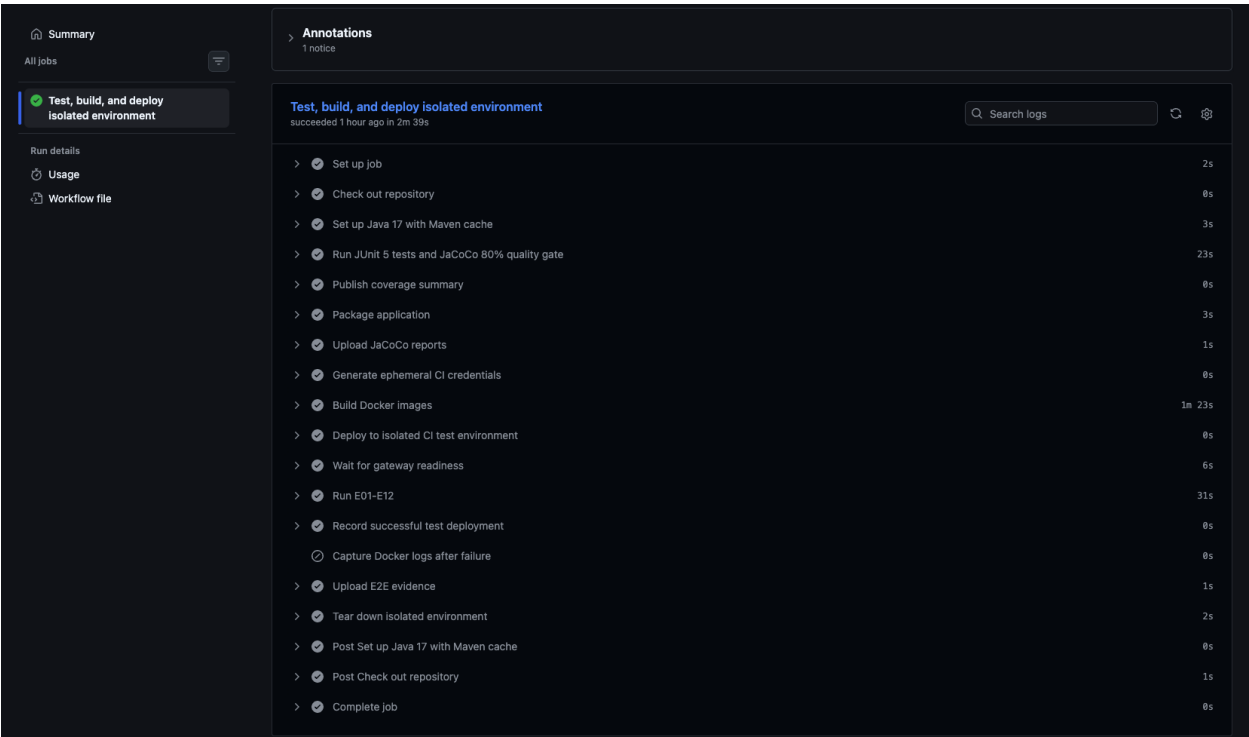


Figura 1. Ejecución de GitHub Actions con pruebas, cobertura, empaquetado, Docker, despliegue, E01-E12, artefactos y teardown en estado exitoso.

3. Pruebas de seguridad y calidad del código

El equipo completó dos análisis complementarios. SonarQube revisó el código fuente, sus métricas y posibles defectos; OWASP ZAP observó la aplicación en ejecución y sus respuestas HTTP. En ambos casos se seleccionó un hallazgo concreto, se interpretó y se aplicó una corrección verificable.

3.1 Análisis completado con SonarQube

Se utilizó SonarQube for IDE durante el desarrollo y SonarQube Community Build 26.9.0.129388 mediante Docker para el análisis global del proyecto Maven multimódulo. El scanner terminó con BUILD SUCCESS y generó un tablero general con métricas de confiabilidad, seguridad, mantenibilidad, cobertura y duplicación.

Métrica	Resultado documentado	Interpretación
Reliability antes	2 bugs; calificación C	Se revisó el hallazgo de AuditFilter.
Reliability después	1 bug restante	El hallazgo seleccionado desapareció después de la corrección.
Vulnerabilities	0; calificación A	No se detectaron vulnerabilidades con las reglas activas.
Security Hotspots	0	No quedaron hotspots pendientes en el análisis.
Code Smells	9; calificación A	Hallazgos de mantenibilidad documentados.
Duplicated Lines	5.8% sobre 1.3k líneas	Métrica global obtenida del tablero.
Coverage en SonarQube	0.0% sobre 384 líneas	JaCoCo no se importó en esa ejecución; la cobertura oficial se documenta por separado.
Código analizado	982 LOC	Alcance reportado por SonarQube.

Hallazgo y corrección

SonarQube señaló que AuditFilter podía lanzar NullPointerException porque getStatusCode() se invocaba dos veces: una para comprobar null y otra para obtener el valor. En un flujo reactivo, una lectura duplicada de un estado potencialmente mutable era innecesaria e insegura. El equipo determinó que no era un falso positivo.

La corrección almacena el resultado en una variable local y evalúa esa única lectura. El segundo análisis confirmó que el hallazgo desapareció. La corrección está presente en el código actual y AuditFilterTest mantiene cuatro pruebas aprobadas.

Antes	Después
Dos llamadas independientes a getStatusCode().	Una llamada almacenada en responseStatus y evaluación segura de null.

La evidencia de SonarQube se considera completada: contiene captura del tablero, métricas, hallazgo seleccionado, interpretación, cambio aplicado y segundo análisis. El reporte corresponde a la versión analizada por SonarQube; los cambios posteriores del repositorio se verificaron mediante JUnit, JaCoCo y CI.

3.2 Pruebas completadas con OWASP ZAP

OWASP ZAP 2.17.0 se ejecutó contra el gateway activo en un entorno Docker local. La actividad fue un análisis pasivo comparativo de GET /health. Se conservaron tanto el reporte inicial como el final, además de una interpretación técnica escrita.

Indicador	Análisis inicial	Análisis final
Respuesta HTTP	200 OK	200 OK
Alertas High	0	0
Alertas Medium	0	0
Alertas Low	1	0
Confianza del hallazgo	Media	No aplica
Total de alertas	1	0

Hallazgo y corrección

ZAP reportó X-Content-Type-Options Header Missing como una alerta Low de confianza Medium, asociada con CWE-693 y OWASP A05:2021 Security Misconfiguration. La ausencia permitía que un navegador intentara deducir el tipo real del contenido mediante MIME sniffing.

Se incorporó SecurityHeadersWebFilter como filtro global de Spring WebFlux. Antes de confirmar la respuesta, establece X-Content-Type-Options: nosniff mediante set, por lo que también sustituye un valor incorrecto enviado por el upstream. Las pruebas de integración validan /health y una ruta proxyficada. Una nueva sesión de ZAP repitió el análisis y no encontró alertas dentro de los parámetros seleccionados.

Evidencia exigida	Material entregado	Estado
Primer escaneo	PDF inicial con una alerta Low.	Incluido
Tabla interpretada	README técnico y comparación dentro de este informe.	Incluida
Cambio en código	SecurityHeadersWebFilter y SecurityHeadersIntegrationTest.	Incluido
Segundo escaneo	PDF final con cero alertas dentro del alcance.	Incluido
Conclusión	Corrección global verificada sin afectar disponibilidad.	Incluida

Alcance interpretado correctamente

Cero alertas significa que el segundo análisis pasivo no encontró problemas en el tráfico observado de /health bajo sus parámetros. No significa que toda la aplicación esté libre de vulnerabilidades. No se ejecutó un active scan, no se exploraron automáticamente rutas JWT o API key y no se probaron inyección, abuso de lógica o disponibilidad. Estas limitaciones no invalidan la actividad: la rúbrica permite analizar endpoints públicos y documentar la cobertura restringida por autenticación.

4. Evaluación de resultados y cierre

El proyecto alcanzó un estado demostrable y reproducible. Los controles de seguridad dejaron de ser ejemplos aislados y quedaron integrados en el flujo principal: las solicitudes atraviesan el gateway, son autenticadas y autorizadas, se someten a límites, generan auditoría y llegan a una API interna que no está expuesta directamente.

Resultados alcanzados

Objetivo	Resultado verificable
Funcionalidad	Registro, login y productos CRUD funcionan sobre la arquitectura real.
Seguridad	JWT, roles, API key heredada, límites, aislamiento, auditoría y nosniff están integrados.
Calidad	JaCoCo exige $\geq 80\%$ por módulo; el artefacto conservado registra 91.26% y 96.36%.
Automatización	GitHub Actions ejecuta pruebas, paquete, Docker, despliegue, E2E, artefactos y limpieza.
Análisis	SonarQube y OWASP ZAP se ejecutaron, interpretaron y produjeron correcciones comprobables.
Evidencia	El entregable conserva reportes, JSON, capturas, workflow y documentación técnica.

Problemas enfrentados y soluciones

Problema	Decisión aplicada	Efecto
La cobertura podía ocultarse en un total agregado.	Gate JaCoCo independiente por módulo.	Ningún módulo compensa al otro.
No había un servidor de prueba permanente.	Despliegue efímero con Docker Compose en GitHub Actions.	Delivery automático y demostrable sin infraestructura adicional.
JWT y API key podían mezclarse.	JWT quedó para productos y API key solo para órdenes heredadas.	Contratos claros y menor ambigüedad.
Sonar detectó lectura insegura del estado HTTP.	Una sola lectura local antes de evaluar null.	Hallazgo eliminado del segundo análisis.
ZAP detectó encabezado faltante.	Filtro global con nosniff y pruebas de integración.	El segundo análisis pasó de una alerta a cero.

Decisiones que fortalecieron la entrega

- Mantener demo-api privada y publicar únicamente el gateway.
- Generar secretos efímeros en CI y usar .env solo como configuración local ignorada.
- Conservar evidencia nativa de herramientas además de resúmenes interpretados.
- Distinguir cobertura JaCoCo de cobertura importada en SonarQube para no presentar métricas engañosas.
- Documentar expresamente las limitaciones de ZAP en lugar de interpretar cero alertas como ausencia total de riesgo.

Limitaciones conocidas

Limitación	Impacto actual	Tratamiento
Datos en memoria	Usuarios y productos se reinician con los contenedores.	Adecuado para el prototipo; documentado en el plan de mejora.
API key compartida	Una sola credencial protege la ruta heredada.	Su alcance está aislado y no sustituye JWT.
Rate limit local	No comparte estado entre varias instancias.	La demostración utiliza una sola instancia.
SonarQube sin JaCoCo importado	El tablero mostró 0.0% de cobertura.	La cobertura se midió y aplicó correctamente mediante JaCoCo por módulo.
ZAP pasivo sobre /health	No cubre rutas autenticadas ni ataques activos.	El alcance está declarado y respaldado por controles manuales/E2E.
Un issue Sonar de Reliability restante	RateLimitPolicy requiere revisión posterior.	No impidió completar ni documentar el análisis realizado.

Aprendizajes del equipo

- Una prueba aprobada no equivale a cobertura suficiente; el porcentaje debe medirse y bloquear el build.
- SAST y DAST son complementarios: SonarQube encontró un defecto de código y ZAP una configuración observable en HTTP.
- La evidencia es más útil cuando enlaza hallazgo, interpretación, cambio y nueva verificación.
- La automatización debe conservar artefactos y limpiar el entorno incluso cuando una etapa falle.
- Los resultados de seguridad deben comunicarse con su alcance para evitar conclusiones que la herramienta no respalda.

Conclusión

ModuShield cumple el objetivo de demostrar un módulo funcional protegido, un proceso CI/CD automatizado y verificaciones de calidad y seguridad interpretadas por el equipo. La solución muestra creación y consulta, control de acceso por roles, rechazo de solicitudes sin credenciales, cobertura superior al mínimo, despliegue de prueba reproducible y correcciones derivadas de SonarQube y OWASP ZAP. La entrega conserva tanto los resultados exitosos como sus límites, lo que permite evaluar el trabajo sin depender de afirmaciones no verificables.

Las acciones posteriores están desarrolladas en PLAN-MEJORA-CONTINUA.pdf, que complementa este cierre con prioridades, horizontes e indicadores medibles.

Inventario de evidencias

Evidencia	Ruta dentro del entregable	Qué demuestra
Pipeline	pipeline/ci.yml	Definición de CI/CD versionada.
Ejecución	pipeline/run-36291416738.json	Metadatos del run exitoso seleccionado.
Cobertura	reportes/pruebas-unitarias/	JaCoCo HTML/XML/CSV, JUnit y ZIP original.
SonarQube	reportes/sonar/Analisis&Evidencia-SonarQube.pdf;	Tablero, métricas, hallazgo y corrección.
ZAP inicial	reportes/seguridad-zap/Modushield - Analisis inicial OWASP ZAP.pdf	Una alerta Low en /health.
ZAP final	reportes/seguridad-zap/ModuShield - Análisis final OWASP ZAP.pdf	Cero alertas dentro del alcance final.
Interpretación ZAP	reportes/seguridad-zap/README.md	Alcance, corrección, comparación y limitaciones.
E2E	evidencias/e2e-ci-36291416738.json	E01-E12 con 12 PASS.
Captura	evidencias/despliegue.png	Secuencia completa del job exitoso.
Plan	PLAN-MEJORA-CONTINUA.pdf	Acciones específicas, métricas, prioridad e innovación.

Referencias técnicas

- Repositorio: <https://github.com/EvenWorseGroup/Modushield>
- Run con evidencia descargada: <https://github.com/EvenWorseGroup/Modushield/actions/runs/36291416738>
- Run posterior al filtro de seguridad: <https://github.com/EvenWorseGroup/Modushield/actions/runs/36311708560>
- Documentación operativa: README.md y docs/ del repositorio.

Fin del informe de cierre